NOTAS:

- **1.-** Si no pones versión y solo dejas el nombre de la imagen se descarga como si pusieses “latest” (ej. ubuntu:latest)
- **2.-** Se pueden borrar contenedores por nombre, pero por id completo o parcial es mejor.

Imágen	Dirección
Nginx	/usr/share/nginx/html/
Apache (httpd)	/usr/local/apache2/htdocs/

COMANDOS VARIOS:
docker pull **[“nombre imagen”:”versión”]** → para descargar imágenes.

docker run **[“nombre imagen”:”versión”]** → para ejecutar una imagen.

docker run -ti **[nombre de la imagen]** bash → para usar una imagen con terminal interactivo.

docker run -d --name **[nombre contenedor] [“nombre imagen”:”versión”]** → para crear un contenedor con una imagen.

docker run -p 8000:80 **[“tu id de Docker Hub”/”nombre imagen”:”versión”]** → para ejecutar una imagen por un puerto de red al que acceder.

docker ps → Para ver los contenedores en marcha.

docker ps -a → Para mostrar todos los contenedores.

docker start **[id del contenedor]**
docker stop **[id del contenedor]**

docker rm **[-f si está starteado] [id del contenedor]** → para borrar contenedores (-f fuerza el borrado aun cuando está activa la imagen).

docker exec **[“nombre imagen”] [“comando a ejecutar”]** → para ejecutar comandos dentro de imagenes que están corriendo (usar docker start antes).

PROCESO PARA CREAR IMAGEN:
mkdir **[nombre directorio]**

cd **[nombre directorio]**

nano Dockerfile → introduces dentro el FROM y el RUN:

```
osboxes@osboxes:~/nginx-netcat$ cat Dockerfile
FROM nginx:1.31.4
RUN apt update && apt install netcat-traditional -y
osboxes@osboxes:~/nginx-netcat$
```

docker build -t **[“nombre imagen”:”versión”]** . → para crear la imagen a partir del Dockerfile (el punto final referencia al Dockerfile del directorio actual).

docker build -t **[“tu id de Docker Hub”/”nombre imagen”:”versión”]** . → para crear una imagen y para que el nombre sea descriptivo y facilite el “docker push”.

docker image rm **[“tu id de Docker Hub”/”nombre imagen”:”versión”]** → para borrar imagenes, puede ser el nombre distinto, siendo siempre el concreto a la imagen que deseas borrar.

docker login → para iniciar sesión en docker hub. Te da un codigo y un enlace para que lo pongas.

docker push **[“tu id de Docker Hub”/”nombre imagen”:”versión”]** → para subir tus imagenes a docker hub (Primero haz docker login).

DOCKERFILE:

- **FROM:** Donde se referencian las imágenes base, las que usamos como base de la imagen que estamos creando. **[“tu id de Docker Hub”/”nombre imagen”:”versión”]**
- **RUN:** Se usa para ejecutar comandos. Su uso típico es para instalar paquetes, compilar codigo, actualizar sistemas operativos...
- **COPY:** Transfiere de la maquina en la que se crea la imagen a la propia imagen archivos.
- **WORKDIR:** Establece un directorio de trabajo para la imagen, será en ese directorio que se ejecutarán distintas ordenes que pongamos, como el RUN.
- **CMD:** Se puede definir una instrucción que se ejecuta en el inicio de cada ejecución de la imagen. **["ejecutable", "param1", "param2"]**
- **USER:** Marca el usuario que ejecuta las instrucciones de RUN o CMD.
- **LABEL:** Sirve para añadir metadatos para poder dejar constancia de quien y cuando creo la imagen.
- **ENV:** Sirve para alterar el valor de una variable de entorno. Añadir antes de otras instrucciones si estas pueden verse afectadas por su estado.

.DOCKERIGNORE:

nano .dockerignore → escribimos dentro los archivos o directorios a no tomar en cuenta.

```
.git/
node_modules/
dist/
```

EJEMPLO:

Ejercicio (Dockerfile):

```
FROM python:3.15

ENV PYTHONUNBUFFERED=1
```

NOTA: sin espacios entre la variable, el igual y el valor.

```
WORKDIR /app
COPY requirements.txt .

RUN pip install -r requirements.txt

COPY oratge.py .

CMD [“python3” , “oratge.py”]
```

Descripción:

Usamos una imagen de base, ponemos como directorio de trabajo /app y le copiamos a este (por el punto final) el archivo requirements.txt. Define un comando para instalar el contenido de requirements.txt que es una version concreta de una librería de python. Luego copiamos oratge.py a /app (igual que antes) y definimos el comando que se ejecutará siempre con la imagen que usa la orden “python3” para ejecutar el archivo “oratge.py”.

Para que en la ejecución los resultado no se guarden en el Buffer y salgan progresivamente, usamos PYTHONUNBUFFERED = 1 para que python, como bien dice el nombre, no use el buffer. El 1 es como escribir True.